

OMI Rule/Algorithm Status Model and KK/KU Canvas Modality

1. Core Split

OMI should use two base spec types:

```
rules
algorithms
```

They answer different questions.

```
Rule:
  What do we know, not know, suspect, or need to discover?

Algorithm:
  What can be computed, implemented, checked, or proved?
```

So the spec has two status systems:

```
Research status:
  Known Known
  Known Unknown
  Unknown Known
  Unknown Unknown

Implementation status:
  Proved
  Implemented
  Defined Model
  Interpretation
```

The first is epistemic.

The second is operational.

They should not be collapsed.

2. Rules as Epistemic Research Objects

A rule belongs to the research-facing model.

It is tagged by the Rumsfeldian quadrants:

```
KK = Known Known  
KU = Known Unknown  
UK = Unknown Known  
UU = Unknown Unknown
```

These are not proof statuses. They are knowledge-orientation statuses.

Known Known — KK

```
Known Known:  
  we know what the object is  
  we know why it matters  
  we know how it relates to the current model
```

Example:

```
512 bits equals 64 bytes.
```

Example:

```
The Omicron NULL Byte Ring closes to zero under XOR.
```

Known Unknown — KU

```
Known Unknown:  
  we know the question  
  we know what is missing  
  we can state the gap
```

Example:

```
We know a representation/refinement proof is needed to connect a C  
implementation to the Coq model.
```

Unknown Known — UK

Unknown Known:

the project already contains working structure or intuition,
but it has not yet been extracted, named, classified, or connected.

Example:

A historical implementation may contain a working resolver pattern before the canon has named it.

Unknown Unknown — UU

Unknown Unknown:

uncharted structure
not yet framed as a question
above the current map

Example:

A future projection relation not yet expressible in the current grammar.

3. Algorithms as Implementation Objects

An algorithm belongs to the implementation-facing model.

It is tagged by execution status:

proved
implemented
defined_model
interpretation

These are not research quadrants. They tell an agent how to treat the object operationally.

Proved

```
proved:  
  active theorem compiles without forbidden assumptions  
  passes the project proof gate
```

Implemented

```
implemented:  
  runtime code or tests demonstrate behavior  
  equivalence to the formal model may still be unproved
```

Defined Model

```
defined_model:  
  terms and decision procedure are precise  
  larger external correspondence is not yet proved
```

Interpretation

```
interpretation:  
  name, image, metaphor, geometry, or reading  
  useful for understanding  
  not promoted to theorem
```

This matches the proof-book status scheme and gives coding agents a safe implementation rail.

4. Two-Layer Status Matrix

Every important OMI object can carry both tags.

```
omi_status:  
  research_modality: KK | KU | UK | UU  
  algorithm_status: proved | implemented | defined_model | interpretation
```

Example:

```
object: Omicron_NULL_Byte_Ring  
kind: algorithm
```

```
research_modality: KK
algorithm_status: proved
```

Example:

```
object: Polyharmonic_Runtime_Governor
kind: rule
research_modality: KU
algorithm_status: defined_model
```

Example:

```
object: historical_unicode_metaverse_renderer
kind: artifact
research_modality: UK
algorithm_status: implemented
```

This prevents the bad collapse:

```
not proved = false
```

Correct reading:

```
not proved = check the research modality and implementation status separately
```

5. Canvas Edge Modality

For `.canvas` or Obsidian graph export, each node can carry four epistemic edges.

Your proposed mapping is good:

```
Top:    UU
Right:  KU
Bottom: KK
Left:   UK
```

I would keep it.

The geometry reads well:

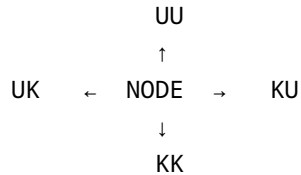
Top / UU:
above the current map
unknown unknown
needs ladder / ascent / discovery

Right / KU:
forward question
known unknown
explicit research gap

Bottom / KK:
foundation
known known
stable base

Left / UK:
latent archive
unknown known
already present but not integrated

So on a standard canvas plane:



Meaning:

Bottom is foundation.
Top is uncharted.
Right is the next stated question.
Left is the latent/archive material that may already contain the answer.

This is stable and intuitive.

6. Why Left = UK and Right = KU Works

Your instinct about left/right is reasonable.

On a standard x/y plane, rightward motion usually feels like forward sequence or explicit next step.

So:

Right = KU

because a Known Unknown is a known research target.

Leftward motion feels like return, archive, memory, or earlier material.

So:

Left = UK

because an Unknown Known is something already present but not yet recognized.

This gives a clean linear sequence:

UK → KK → KU → UU

as a research progression:

```
latent archive
→ extracted foundation
→ explicit question
→ frontier beyond current questions
```

But spatially the node itself is the current checkpoint.

7. Canvas Node Schema

A canvas node can be structured as:

```
{
  "id": "omi.projection.diagram",
  "type": "omi-node",
  "label": "Omi-Projection Diagram",
  "kind": "construct",
  "grammar": {
    "subject": "Omi",
    "predicate": "Projection",
    "object": "Diagram"
  }
}
```

```

},
"axes": {
  "cardinality": "finite declared state set",
  "chirality": "forward/inverse traversal",
  "ordinality": "declared transition order",
  "modality": "impotent or innocuous projection"
},
"status": {
  "research_modality": "KK",
  "algorithm_status": "defined_model"
},
"edges": {
  "top_UU": [],
  "right_KU": [],
  "bottom_KK": [],
  "left_UK": []
}
}

```

Each edge direction means:

```

{
  "top_UU": "unknown unknown / above the current map",
  "right_KU": "known unknown / explicit research gap",
  "bottom_KK": "known known / stable foundation",
  "left_UK": "unknown known / latent archive or unintegrated artifact"
}

```

8. Canvas Edge Schema

```

{
  "id": "edge-001",
  "fromNode": "omi.projection.diagram",
  "toNode": "omi.latin.resolver",
  "side": "right",
  "modality": "KU",
  "meaning": "Projection Diagram needs a stricter resolver declaration model.",
  "axis_focus": "modality"
}

```

Edges should also identify their face focus:


```
cardinality
chirality
ordinality
functionality
```

You said functionality here, and that is better than modality for the pyramid face.

So the project can use both:

```
modality:
  epistemic edge mode, KK/KU/UK/UU

functionality:
  what the node/edge does in execution
```

That prevents word collision.

9. Pascal Pyramid Assurance Model

The pyramid image is strong.

The foundation is KK.

Above it, KU and UK form the ladder and retrieval faces.

UU sits above as frontier.

The four faces of the pyramid can focus on:

```
cardinality
chirality
ordinality
functionality
```

So every block has two classifications:

```
edge modality:
  KK / KU / UK / UU
```

```
face focus:
  cardinality / chirality / ordinality / functionality
```

Meaning:

```
assurance_block:
  epistemic_modality: KK
  face_focus: cardinality
```

Examples:

```
- block: "512-bit runtime packet"
  epistemic_modality: KK
  face_focus: cardinality

- block: "Tetragrammatron Governor reassociation"
  epistemic_modality: KU
  face_focus: functionality

- block: "old Unicode metaverse renderer"
  epistemic_modality: UK
  face_focus: functionality

- block: "future higher-dimensional projection"
  epistemic_modality: UU
  face_focus: chirality
```

10. Rules vs Algorithms in YAML

```
omi_spec_unit:
  id: omi.rule.address_identity
  base_type: rule
  form: one_sentence
  content: "Address is identity."

research:
  modality: KK
  confidence: foundation

axes:
  cardinality: "one addressed identity per declared coordinate"
  chirality: "OMI/IMO inverse pairing"
```

```
ordinality: "identity precedes interpretation"
functionality: "object selection"
```

```
linked_algorithms:
- omi.algorithm.null_ring_closure
- omi.algorithm.address256_self_cancels
```

Algorithm object:

```
omi_spec_unit:
  id: omi.algorithm.null_ring_closure
  base_type: algorithm
  form: deterministic_procedure
  content: "Reduce adjacent XOR edge witnesses over a closed byte ring."

  implementation:
    status: proved
    proof_module: coq/02-closure/NullRingCloses.v

  axes:
    cardinality: "4 byte-ring points"
    chirality: "closed forward/inverse cycle"
    ordinality: "00 -> 20 -> 7F -> FF -> 00"
    functionality: "closure verification"

  supports_rules:
    - omi.rule.address_identity
```

This is the best split:

```
rules carry epistemic modality
algorithms carry implementation status
```

11. Rule/Algorithm Pairing

Each rule should eventually map to one or more algorithms.

```
Rule:
  Address is identity.

Algorithms:
  address256_self_cancels
```

```
null_ring_closure
omnicron_envelope_valid
```

Rule:
Projection does not command application.

Algorithms:
projection_gate
authority_pipeline_preserves_decision
metatron_cannot_make_unlawful_lawful

Rule:
Any two coordinates determine the third.

Algorithms:
latin_resolver_lookup
quasigroup_recovery
orthogonal_array_pair_recovery

A rule without an algorithm is not bad.

It is a research object.

An algorithm without a rule is not bad.

It is an implementation object awaiting canonical naming.

12. Canvas Front Matter Shape

```
omi_canvas_model:
  title: OMI Epistemic/Algorithmic Canvas
  node_semantics:
    center: current_checkpoint
    top: UU
    right: KU
    bottom: KK
    left: UK

  edge_modalities:
    UU:
      side: top
      meaning: unknown_unknown
```

```

    role: frontier_above_current_map
KU:
    side: right
    meaning: known_unknown
    role: explicit_question_or_gap
KK:
    side: bottom
    meaning: known_known
    role: foundation
UK:
    side: left
    meaning: unknown_known
    role: latent_archive_or_unintegrated_artifact

face_focus:
- cardinality
- chirality
- ordinality
- functionality

base_types:
  rule:
    purpose: epistemic_research_model
    statuses:
      - KK
      - KU
      - UK
      - UU
  algorithm:
    purpose: implementation_execution_model
    statuses:
      - proved
      - implemented
      - defined_model
      - interpretation

```

13. Canon Statement

OMI should distinguish rules from algorithms.

Rules are epistemic research objects. They are classified by Known Known, Known Unknown, Unknown Known, and Unknown Unknown.

Algorithms are implementation objects. They are classified by Proved, Implemented, Defined Model, and Interpretation.

In Omi-Canvas, each node may carry epistemic edges:

```
Top    = UU
Right  = KU
Bottom = KK
Left   = UK
```

The bottom edge gives the foundation. The right edge gives the explicit research gap. The left edge gives latent archive material not yet integrated. The top edge gives the frontier above the current map.

Each block also identifies one face focus:

```
cardinality
chirality
ordinality
functionality
```

Together, the canvas forms a Pascal-pyramid-like assurance model: stable known foundations at the base, explicit questions as ladders, latent archive knowledge as recoverable side support, and unknown unknowns above the current map.

This gives OMI both an epistemic research grammar and an implementation execution grammar without confusing one for the other.